

A* Pathfinding Algorithm on Occupancy Grids

A Comprehensive Lecture & Coding Guide
From Theory to Implementation

Authored by

Robotics and Artificial Intelligence Nigeria (RAIN)

Department of Robot Development and Automation (RDA)

Course Information	
Organization	Robotics and Artificial Intelligence Nigeria (RAIN)
Department	Robot Development and Automation (RDA)
Prerequisites	Basic Python, arrays/matrices, introductory algorithms
Duration	3–4 hours (lecture + guided coding)
Libraries	NumPy, OpenCV (cv2), Matplotlib
Topics	Occupancy grids, graph search, A*, heuristics, came_from, path reconstruction

Table of Contents

1. Part 1: Motivation — Why Do We Need Pathfinding?
2. Part 2: Representing the World — Occupancy Grids
3. Part 3: Graph Search Fundamentals
4. Part 4: The A* Algorithm — Deep Dive
5. Part 5: Path Reconstruction & the `came_from` Dictionary
6. Part 6: Visualization with Matplotlib
7. Part 7: What Happens When Obstacles Block the Heuristic?
8. Part 8: Full Worked Example — Step-by-Step A* Trace with `came_from`
9. Part 9: When Can g-scores Decrease?
10. Part 10: Performance Analysis & Improvements
11. Part 11: Complete Program (Copy-Paste Ready)
12. Part 12: Exercises — Build It Yourself
13. Part 13: Summary & Key Takeaways
14. Appendix A: Required Libraries
15. Appendix B: Creating Test Images
16. Appendix C: PriorityQueue Quick Reference

Part 1: Motivation — Why Do We Need Pathfinding?

Imagine you are building a mobile robot that needs to navigate from one end of a warehouse to another. The warehouse has shelves, walls, and other obstacles. The robot has a map — but how does it decide **which route to take**?

This is the **pathfinding problem**: given a map with obstacles, find a collision-free route from a start location to a goal location, ideally the shortest one.

Real-world applications:

- Mobile robot navigation (warehouse robots, delivery drones)
- Video game AI (NPCs navigating terrain)
- Self-driving vehicles (route planning)
- Evacuation route planning
- Drone flight path planning

Part 2: Representing the World — Occupancy Grids

2.1 What Is an Occupancy Grid?

An occupancy grid is a 2D array (matrix) where each cell represents a small region of space. Each cell holds one of two values:

- **1 (white / free):** The cell is empty — the robot can pass through it.
- **0 (black / occupied):** The cell is blocked — there is an obstacle.

Think of it like graph paper laid over a floor plan. Each square is either "passable" or "blocked."

2.2 Creating an Occupancy Grid from an Image

In practice, maps often come as images (satellite photos, floor plan scans, game level designs). We need to convert these images into binary grid format using this pipeline:

Image (grayscale) → Thresholding → Binary Grid (0s and 1s)

Step-by-step logic:

- **Step 1 — Load the image in grayscale.** Each pixel has a value from 0 (black) to 255 (white).
- **Step 2 — Apply a threshold.** Pixels brighter than the threshold become 255 (free); darker ones become 0 (blocked).
- **Step 3 — Normalize to 0 and 1.** Divide all values by 255 so the grid contains only 0s and 1s.

2.3 Code — create_occupancy_grid_from_image

```
import cv2
import numpy as np

def create_occupancy_grid_from_image(image_path, threshold=128):
    # Step 1: Load image as grayscale (single channel, 0-255)
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    # Step 2: Apply binary threshold
    # Pixels > threshold become 255, pixels <= threshold become 0
    _, thresholded = cv2.threshold(image, threshold, 255,
                                   cv2.THRESH_BINARY)

    # Step 3: Normalize to 0 and 1
    occupancy_grid = thresholded // 255

    return occupancy_grid, image
```

2.4 Key Functions

Function	Purpose
<code>cv2.imread(path, cv2.IMREAD_GRAYSCALE)</code>	Load image as 2D NumPy array (0-255)
<code>cv2.threshold(image, thresh, maxval, type)</code>	Convert grayscale to binary using a cutoff

Note: Threshold tuning: Lower threshold = more free space (permissive). Higher = less free space (conservative). 128 is the midpoint default.

Part 3: Graph Search Fundamentals

3.1 The Grid as a Graph

An occupancy grid is implicitly a **graph**. Every free cell (value = 1) is a **node**. Each node connects to its 4 cardinal neighbors (up, down, left, right) if they are also free — these connections are **edges**. This is called **4-connectivity**.

The 4 possible moves from any cell (x, y):

```
(x+1, y) → move right
(x-1, y) → move left
(x, y+1) → move down
(x, y-1) → move up

# In code:
directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
```

3.2 Core Data Structures for Search

Structure	Purpose
Open set (frontier)	Nodes discovered but not yet explored
Came-from map	Records which node we arrived from (for path reconstruction)
Cost map (g-score)	The cheapest known cost to reach each node from start

3.3 The Algorithm Family Tree

To understand A*, it helps to see how it relates to other search algorithms:

Algorithm	Picks next node by	Optimal?	Efficient?
BFS	FIFO queue order	Yes (uniform)	No
Dijkstra	Lowest actual cost $g(n)$	Yes	Medium
Greedy Best-First	Lowest estimated $h(n)$	No	Fast but unreliable
A*	Lowest $g(n) + h(n)$	Yes	Yes — best of both

Key Insight: A* combines the best of both worlds: it uses actual cost (like Dijkstra) AND a heuristic estimate of remaining distance (like Greedy), making it both **optimal** and **efficient**.

Part 4: The A* Algorithm — Deep Dive

4.1 The Core Formula

A* evaluates every node using the score **f(n)**:

$$f(n) = g(n) + h(n)$$

- **g(n)** = actual cost of the cheapest path found from **start** → **n** so far
- **h(n)** = heuristic *estimate* of the cost from **n** → **goal**
- **f(n)** = estimated total cost of the best path **through n**

The node with the **lowest f-score** gets explored next. This is why A* uses a **priority queue** (min-heap) — it always picks the most promising node.

4.2 The Heuristic: Manhattan Distance

For 4-directional movement on a grid, we use the **Manhattan distance**:

$$h(n) = |n.x - goal.x| + |n.y - goal.y|$$

```
def heuristic(cell, goal):  
    return abs(cell[0] - goal[0]) + abs(cell[1] - goal[1])
```

Why Manhattan distance? With only horizontal and vertical moves, you cannot reach the goal in fewer steps than $|dx| + |dy|$. So this heuristic **never overestimates** the true cost. This property is called **admissibility**.

Important: If the heuristic overestimates, A* might skip the optimal path because it looks "too expensive." An admissible heuristic guarantees A* always finds the shortest path.

Other heuristics for reference:

Heuristic	Formula	Best for
Manhattan	$ dx + dy $	4-directional movement
Euclidean	$\sqrt{dx^2 + dy^2}$	Diagonal movement allowed
Chebyshev	$\max(dx , dy)$	8-directional, uniform diag cost
$h(n) = 0$	Always zero	Degenerates to Dijkstra

4.3 The A* Pseudocode

```
INITIALIZE:  
    open_set ← priority queue with (f=0, start)  
    came_from ← empty dictionary  
    g_score[all cells] ← infinity  
    g_score[start] ← 0  
  
LOOP:  
    while open_set is not empty:  
        current ← pop node with lowest f-score
```

```

    if current == goal:
        RETURN reconstruct_path(came_from, current)

    for each neighbor of current:
        if neighbor is in-bounds AND free (value=1):
            tentative_g = g_score[current] + 1

            if tentative_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                f = tentative_g + heuristic(neighbor, goal)
                add (f, neighbor) to open_set

    RETURN None (no path exists)

```

4.4 Complete Python Code

```

from queue import PriorityQueue
import numpy as np

def a_star_search(occupancy_grid, start, goal):

    def heuristic(cell, goal):
        return abs(cell[0] - goal[0]) + abs(cell[1] - goal[1])

    open_set = PriorityQueue()
    open_set.put((0, start)) # (f-score, node)

    came_from = {}

    g_score = {cell: float('inf')
                for cell in np.ndindex(occupancy_grid.shape)}
    g_score[start] = 0

    while not open_set.empty():
        _, current = open_set.get()

        if current == goal:
            return reconstruct_path(came_from, current)

        for dx, dy in [(1,0), (-1,0), (0,1), (0,-1)]:
            x, y = current[0]+dx, current[1]+dy

            if (0 <= x < occupancy_grid.shape[1] and
                0 <= y < occupancy_grid.shape[0] and
                occupancy_grid[y, x] == 1):

                tentative_g = g_score[current] + 1

                if tentative_g < g_score[(x, y)]:
                    came_from[(x, y)] = current
                    g_score[(x, y)] = tentative_g
                    f = tentative_g + heuristic((x,y), goal)
                    open_set.put((f, (x, y)))

    return None

```

4.5 Coordinate Convention — The #1 Bug Source

Important: Cells are stored as (x, y) where x = column and y = row. But NumPy arrays are indexed as `array[row, col] = array[y, x]`. That is why the obstacle check is `occupancy_grid[y, x]` (not `occupancy_grid[x, y]`), and bounds use `shape[1]` for x-limit (columns) and `shape[0]` for y-limit (rows). **If your path goes through walls, this is almost certainly the bug.**

Part 5: Path Reconstruction & the came_from Dictionary

5.1 How came_from Works

Every time A* finds a better path to a neighbor, it records the parent:

```
came_from[(x, y)] = current
```

After the search completes, came_from is a chain of breadcrumbs from goal back to start. The start node has no entry (it has no parent).

5.2 The Reconstruction Code

```
def reconstruct_path(came_from, current):
    path = [current]          # Start at the goal
    while current in came_from: # Follow chain backwards
        current = came_from[current]
        path.insert(0, current) # Prepend each parent
    return path               # Result: [start, ..., goal]
```

Performance improvement: path.insert(0, x) is $O(n)$ per call, making reconstruction $O(n^2)$. For large paths, append and reverse:

```
def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current) # 0(1) amortized
    path.reverse()          # Single 0(n) pass
    return path
```

5.3 came_from Is a Tree, Not a Path

This is a crucial conceptual point. The came_from dictionary is **not** a single path. It is a **tree** — a collection of breadcrumbs covering every cell A* ever found a route to. Many of those breadcrumbs lead to dead ends or suboptimal branches.

Three critical rules about came_from:

- **Entries are only added, never deleted.** When A* hits a dead end, it does not go back and remove the entries it just created. They stay in the dictionary forever.
- **Entries can be overwritten** (but only with a better parent). If A* discovers a cheaper path to a cell, it updates came_from to point to the new parent. But this is an overwrite, not a deletion.
- **Reconstruction ignores irrelevant entries.** Since we start at the goal and follow the chain backwards, we only touch entries that are on the actual optimal path. Dead end entries are simply never accessed.

The full worked example in Part 8 shows came_from building up iteration by iteration, including what happens at the dead end — pay close attention to how entries for (1,0) and (2,0) persist even after A* detours around them.

Part 6: Visualization with Matplotlib

6.1 Displaying the Grid

```
import matplotlib.pyplot as plt

plt.imshow(occupancy_grid, cmap='gray', origin='lower')
plt.colorbar()
plt.grid(True)
plt.show()
```

Note: `origin='lower'` is crucial. Without it, row 0 appears at the top (image convention). Setting it to 'lower' puts row 0 at the bottom, matching the conventional x-y coordinate system where y increases upward.

6.2 Overlaying Start, Goal, and Path

```
# Start (green circle) and Goal (red circle)
plt.plot(start[0], start[1], 'go', label='Start')
plt.plot(goal[0], goal[1], 'ro', label='Goal')

# Path (blue line with dots)
path_x = [cell[0] for cell in path_found]
path_y = [cell[1] for cell in path_found]
plt.plot(path_x, path_y, marker='o', markersize=4,
         color='blue', label='Path')

plt.legend()
plt.title('A* Path on Occupancy Grid')
plt.show()
```

Part 7: What Happens When Obstacles Block the Heuristic?

This is one of the most important conceptual questions about A*: if the heuristic $h(n)$ computes distance as if there were no walls, how does A* navigate around obstacles?

7.1 $h(n)$ ALWAYS Ignores Obstacles

The heuristic is "blind" to obstacles. It always computes the straight-line Manhattan distance as if the grid were completely empty. Wall in the way? It pretends it is not there.

```
# h(n) does NOT look at the grid. Ever.
def heuristic(cell, goal):
    return abs(cell[0] - goal[0]) + abs(cell[1] - goal[1])
```

If the current cell is at (2, 2) and the goal is at (8, 2), $h(n)$ returns 6 — even if there is a massive wall between them that forces a 20-step detour.

7.2 Why This Still Works: Admissibility

An admissible heuristic must satisfy one rule: **never overestimate the true cost**. Obstacles can only make the real path *longer* than the straight-line distance, never shorter. So Manhattan distance is always an underestimate when obstacles are present:

Without obstacle:	true cost = 6,	$h(n) = 6$	✓ (exact)
With obstacle:	true cost = 20,	$h(n) = 6$	✓ (underestimate)

Both cases satisfy $h(n) \leq \text{true cost}$, so A* remains optimal.

7.3 The Real Navigation Mechanism

Key Insight: $h(n)$ is NOT the part that navigates around obstacles. **$g(n)$ is.** The two scores play completely different roles. The heuristic provides direction (a "compass"), while $g(n)$ tracks the actual cost of detours.

Here is what actually happens when A* encounters a wall:

- **Step 1:** A* heads toward the goal (h pulls it in that direction).
- **Step 2:** It hits the wall. Those cells have occupancy = 0, so they **fail the** `occupancy_grid[y, x] == 1` **check** and are never added to the open set.
- **Step 3:** The only unexplored nodes left are ones going *around* the wall. Their g -scores are higher (longer path so far), but they are the only options.
- **Step 4:** A* follows those detour nodes. The h component still pulls toward the goal, so it naturally finds its way around and back toward the target.

A* does not "plan" the detour in advance. It **discovers** it one cell at a time, guided by the interplay of g (cost so far) and h (estimated remaining cost).

7.4 The Efficiency Consequence

When obstacles are present, the gap between $h(n)$ and the true remaining cost widens. A* becomes less efficient (explores more nodes) because the heuristic is less informative:

No obstacles: $h(n) \approx \text{true cost} \rightarrow \text{A* goes almost straight}$
(very few nodes explored)

Many obstacles: $h(n) \ll \text{true cost} \rightarrow \text{A* explores more broadly,}$
behaves closer to Dijkstra

In the extreme case where $h(n) = 0$ for every node, A* degenerates into Dijkstra's algorithm — still optimal, but exploring in all directions without any compass toward the goal.

Part 8: Full Worked Example — Step-by-Step A* Trace with came_from

This is the most important section of the lecture. We will trace A* step by step on a grid that forces a detour, so you can see exactly how the algorithm navigates around obstacles.

8.1 The Grid

	c0	c1	c2	c3	c4	c5	c6
r0:	S	.	.	X	.	.	G
r1:	.	X	X	X	X	X	.
r2:	.	.	.	.	.	.	.
r3:	.	.	.	X	X	X	.
r4:	.	.	.	.	.	.	.

S = (0,0) and G = (6,0) using (x, y) = (column, row) convention.

Heuristic: $h(x, y) = |x - 6| + |y - 0| = |x - 6| + y$

Row 1 is almost completely blocked, so the direct path is impossible. A* must go down to row 2, cross, then come back up.

8.2 Initialization

```
open_set: [(f=6, (0,0))]
g_score: (0,0) = 0, everything else = infinity
came_from: {} ← empty dictionary
```

8.3 The Search — Iteration by Iteration

We now trace every iteration, showing how **came_from** builds up alongside the open set. Pay close attention to what happens at the dead end (Iteration 3) — notice that **came_from** is **never modified or cleaned up** when the algorithm detours.

Iteration 1 — Expand (0,0). $\alpha=0$

```
Neighbor checks:
(1,0): free. g=0+1=1. 1<inf? YES h=|1-6|+0=5 f=6 → ADD
(0,1): free. g=0+1=1. 1<inf? YES h=|0-6|+1=7 f=8 → ADD
(-1,0): out of bounds → SKIP
(0,-1): out of bounds → SKIP
```

```
open_set: [(6,(1,0)), (8,(0,1))]
```

```
came_from: {
  (1,0): (0,0),      "to reach (1,0), I came from start"
  (0,1): (0,0),      "to reach (0,1), I came from start"
}
```

Iteration 2 — Expand (1,0). $\alpha=1$ lowest f=61

```
Neighbor checks:
(2,0): free. g=1+1=2. 2<inf? YES h=|2-6|=4 f=6 → ADD
(0,0): free. g=1+1=2. 2<0? NO → SKIP (already better)
```

```
(1,1): BLOCKED (wall)           → SKIP
(1,-1): out of bounds           → SKIP
```

```
open_set:  [(6,(2,0)), (8,(0,1))]
```

```
came_from: {
  (1,0): (0,0),
  (0,1): (0,0),
  (2,0): (1,0),      ← NEW
}
```

Iteration 3 — Expand (2.0). $\alpha=2$ flowest f=61 ★ DEAD END

```
Neighbor checks:
(3,0): BLOCKED (wall)       → SKIP
(1,0): g=2+1=3. 3<1? NO    → SKIP
(2,1): BLOCKED (wall)       → SKIP
(2,-1): out of bounds       → SKIP
```

```
open_set:  [(8,(0,1))]      ← only one option left!
```

```
came_from: {
  (1,0): (0,0),             ← DEAD END ENTRY - still here!
  (0,1): (0,0),
  (2,0): (1,0),             ← DEAD END ENTRY - still here!
}                             ← NO entries were deleted
```

Key Insight: DEAD END! A* hit the wall at column 3. No new nodes were added. But look at came_from — the entries for (1,0) and (2,0) are **still there**. Nothing was deleted! A* does not clean up dead end breadcrumbs. It simply moves on to the next node in the queue: (0,1). **This is the moment A* is forced to detour.**

Iteration 4 — Expand (0.1). $\alpha=1$ fonly option. f=81

```
Neighbor checks:
(1,1): BLOCKED             → SKIP
(0,0): g=2. 2<0? NO        → SKIP
(0,2): free. g=2. h=|0-6|+2=8 f=10 → ADD
(-1,1): out of bounds       → SKIP
```

```
open_set:  [(10,(0,2))]
```

```
came_from: {
  (1,0): (0,0),             ← dead branch, just sitting here
  (0,1): (0,0),
  (2,0): (1,0),             ← dead branch, just sitting here
  (0,2): (0,1),             ← NEW: the detour begins!
}
```

A* is now moving downward into the open area of row 2. The dead end entries remain untouched.

Iteration 5 — Expand (0.2). $\alpha=2$ f=101

```
Neighbor checks:
(1,2): free. g=3. h=|1-6|+2=7 f=10 → ADD
(0,3): free. g=3. h=|0-6|+3=9 f=12 → ADD
(0,1): g=3. 3<1? NO        → SKIP
```

(-1,2): out of bounds → SKIP

open_set: [(10, (1,2)), (12, (0,3))]

```
came_from: {
  (1,0): (0,0),          ← dead branch
  (0,1): (0,0),          ← dead branch
  (2,0): (1,0),          ← dead branch
  (0,2): (0,1),
  (1,2): (0,2),          ← NEW
  (0,3): (0,2),          ← NEW (side branch)
}
```

Note: (0,3) has f=12 — it goes deeper away from the goal. A* prefers (1,2) with f=10 because it moves toward the goal. **This is h doing its compass job.**

Iteration 6 — Expand (1.2). α=3 f=101

Neighbor checks:

```
(2,2): free. g=4. h=|2-6|+2=6   f=10 → ADD
(1,3): free. g=4. h=|1-6|+3=8   f=12 → ADD
(0,2): g=4. 4<2? NO             → SKIP
(1,1): BLOCKED                  → SKIP
```

came_from now has 8 entries:

```
(1,0):(0,0) (0,1):(0,0) (2,0):(1,0) (0,2):(0,1)
(1,2):(0,2) (0,3):(0,2) (2,2):(1,2) (1,3):(1,2)
                        ↑ NEW      ↑ NEW
```

Iteration 7 — Expand (2.2). α=4 f=101

```
(3,2): free. g=5. h=|3-6|+2=5   f=10 → ADD
(2,3): free. g=5. h=|2-6|+3=7   f=12 → ADD
(1,2): α=5. 5<3? NO → SKIP      (2,1): BLOCKED → SKIP
```

came_from adds: (3,2):(2,2) and (2,3):(2,2)

Total entries: 10

Iteration 8 — Expand (3.2). α=5 f=101

```
(4,2): free. g=6. f=10 → ADD
(3,3): BLOCKED (3,1): BLOCKED (2,2): 6<4? NO → SKIP
```

came_from adds: (4,2):(3,2) Total: 11

Iteration 9 — Expand (4.2). α=6 f=101

```
(5,2): free. g=7. f=10 → ADD
(4,3): BLOCKED (4,1): BLOCKED (3,2): 7<5? NO → SKIP
```

came_from adds: (5,2):(4,2) Total: 12

Iteration 10 — Expand (5.2). α=7 f=101

```
(6,2): free. g=8. f=10 → ADD
(5,1): BLOCKED (5,3): BLOCKED (4,2): 8<6? NO → SKIP
```

came_from adds: (6,2):(5,2) Total: 13

We have reached the right edge of row 2. Now A* heads upward.

Iteration 11 — Expand (6.2). $\alpha=8$ $f=10$

```
(6,1): free. g=9. h=1   f=10 → ADD
(6,3): free. g=9. h=3   f=12 → ADD
(5,2):  $\alpha=9$ .  $9<7?$  NO → SKIP   (7,2): out of bounds
```

came_from adds: (6,1):(6,2) and (6,3):(6,2) Total: 15

Iteration 12 — Expand (6.1). $\alpha=9$ $f=10$

```
(6,0): free. g=10. h=0   f=10 → ADD    ★ This is the GOAL!
(5,1): BLOCKED    (6,2):  $10<8?$  NO    (7,1): out of bounds
```

came_from adds: (6,0):(6,1) Total: 16

Iteration 13 — Pop (6,0)... GOAL REACHED!

(6,0) is the goal! A* terminates and reconstructs the path.

8.4 The Complete came_from Dictionary at Goal

Here is the **entire** came_from dictionary at the moment A* reaches the goal. Notice it contains entries for **every cell A* ever found a path to** — including dead ends and side branches:

```
came_from = {
    (1,0): (0,0),      ← DEAD END (part of the failed rightward push)
    (0,1): (0,0),      ← ON THE OPTIMAL PATH
    (2,0): (1,0),      ← DEAD END (part of the failed rightward push)
    (0,2): (0,1),      ← ON THE OPTIMAL PATH
    (1,2): (0,2),      ← ON THE OPTIMAL PATH
    (0,3): (0,2),      ← SIDE BRANCH (never led anywhere useful)
    (2,2): (1,2),      ← ON THE OPTIMAL PATH
    (1,3): (1,2),      ← SIDE BRANCH
    (3,2): (2,2),      ← ON THE OPTIMAL PATH
    (2,3): (2,2),      ← SIDE BRANCH
    (4,2): (3,2),      ← ON THE OPTIMAL PATH
    (5,2): (4,2),      ← ON THE OPTIMAL PATH
    (6,2): (5,2),      ← ON THE OPTIMAL PATH
    (6,3): (6,2),      ← SIDE BRANCH
    (6,1): (6,2),      ← ON THE OPTIMAL PATH
    (6,0): (6,1),      ← GOAL — ON THE OPTIMAL PATH
}
```

16 entries total. Only 10 are on the optimal path. The other 6 are dead ends and side branches that **will never be looked at again**.

8.5 Path Reconstruction — How Dead Ends Are Ignored

Reconstruction starts at the **goal** and follows the came_from chain **backwards**. It only touches entries that form a direct chain from goal to start:

```
Start at (6,0). Look up came_from[(6,0)] → (6,1)
                Look up came_from[(6,1)] → (6,2)
                Look up came_from[(6,2)] → (5,2)
                Look up came_from[(5,2)] → (4,2)
                Look up came_from[(4,2)] → (3,2)
```

```

Look up came_from[(3,2)] → (2,2)
Look up came_from[(2,2)] → (1,2)
Look up came_from[(1,2)] → (0,2)
Look up came_from[(0,2)] → (0,1)
Look up came_from[(0,1)] → (0,0)
(0,0) not in came_from → STOP (start)

```

Key Insight: The reconstruction **never touched** entries (1,0), (2,0), (0,3), (1,3), (2,3), or (6,3). Those entries sit in the dictionary harmlessly. The chain from goal to start simply does not pass through them. **No deletion was needed. No cleanup. No reconstruction of came_from during the detour.**

8.6 The Final Path Visualized

	c0	c1	c2	c3	c4	c5	c6	
r0:	[S]	.	.	X	.	.	[G]	↑ step 10
r1:	[↓]	X	X	X	X	X	[↑]	step 1 down, step 9 up
r2:	[→]	[→]	[→]	[→]	[→]	[→]	[↑]	steps 2-8 across
r3:	.	.	.	X	X	X	.	
r4:	.	.	.	.	.	.	.	

Path length: 10 steps

8.7 Observations from This Example

Key Insight: Every node along the optimal path had **f = 10**. This is not a coincidence — the heuristic was accurate enough that the estimated total cost stayed constant along the best route. A* marched straight along this path without wasting time.

Key Insight: Nodes like (0,3), (1,3), (2,3), (6,3) were added to the queue with f=12 but **never expanded**. A* found the goal before needing them. Rows 3 and 4 were **completely ignored**. A less informed algorithm like BFS would have explored all of them.

Key Insight: **came_from is NOT a path — it is a tree.** It contains breadcrumbs to every cell A* ever discovered a path to, including dead ends and side branches. Reconstruction works because it starts at the goal and follows only the one chain that leads back to the start — all other entries are harmlessly ignored. Nothing is ever deleted from came_from.

Part 9: When Can g-scores Decrease?

A common and very logical question: since we are always adding to g as we explore, how can a g -score ever *decrease*? Every step forward adds 1 to g , so going backwards from a later cell to an earlier one always proposes a worse g -score.

9.1 Why Going Backwards Gives a Worse g

Consider what happens when we expand (2,0) in our example and look at neighbor (1,0):

```
We are at (2,0), which has g = 2.  
Neighbor (1,0): tentative_g = 2 + 1 = 3  
But g_score[(1,0)] is already 1 (set when Start discovered it).
```

```
Check: 3 < 1? NO. So nothing happens.
```

```
We would be saying: "I found a path to (1,0) costing 3"  
when we already know a path costing 1. Pointless.
```

The stored g -score of (1,0) never changes — it stays at 1. The code is comparing: "Would going backwards give us a cheaper path?" The answer is no.

9.2 When g -scores CAN Decrease

The g -score decreases when a **completely different branch of exploration** reaches the same cell through a shorter route. This happens in complex mazes with multiple paths to the same location.

Think of two scouts exploring different hallways of a maze. The first scout reaches a room and reports "it took me 12 steps." The second scout finds a shortcut and says "I got there in 7." We update our records: $7 < 12$ is true, so g -score drops to 7.

This becomes much more common with **non-uniform costs** — for example, if some cells are "mud" (cost 3), some are "road" (cost 1), and some are "highway" (cost 0.5). Then A* might first reach a cell through expensive mud, and later discover a cheaper highway route.

9.3 The Bottom Line

Key Insight: For simple uniform-cost grids, g -scores rarely decrease in practice. Your intuition that " g always increases as we explore" is basically right. The `if tentative_g < g_score` check is a **safety net** that makes A* correct in **all** situations, including complex mazes and non-uniform costs.

Part 10: Performance Analysis & Improvements

10.1 Complexity

Aspect	Complexity	Notes
Time (worst)	$O(b^d)$	b = branching factor (4), d = path depth
Time (practical)	Much < BFS	Heuristic prunes the search space
Space	$O(N)$	N = total cells in the grid

10.2 Improvement: Use defaultdict for g_score

The original code pre-allocates `g_score` for every cell. For a 400×400 image, that is 160,000 dictionary entries before the search even begins.

```
# BEFORE (wasteful):
g_score = {cell: float('inf')}
for cell in np.ndindex(occupancy_grid.shape):

# AFTER (efficient):
from collections import defaultdict
g_score = defaultdict(lambda: float('inf'))
g_score[start] = 0
# Entries created only when accessed
```

10.3 Improvement: Add a Closed Set

Without tracking visited nodes, the same node may be popped and expanded multiple times:

```
closed_set = set()

while not open_set.empty():
    _, current = open_set.get()

    if current in closed_set:
        continue # Already fully explored
    closed_set.add(current)

    if current == goal:
        return reconstruct_path(came_from, current)
    # ... rest of loop
```

10.4 Extension: Diagonal Movement

```
import math

# 8-directional movement
directions = [
    (1,0), (-1,0), (0,1), (0,-1), # cardinal: cost 1.0
    (1,1), (1,-1), (-1,1), (-1,-1) # diagonal: cost sqrt(2)
]

for dx, dy in directions:
    step_cost = math.sqrt(dx**2 + dy**2) # 1.0 or ~1.414
```

```
tentative_g = g_score[current] + step_cost
```

Note: When allowing diagonals, switch the heuristic from Manhattan to Euclidean or Chebyshev distance to maintain admissibility.

Part 11: Complete Program (Copy-Paste Ready)

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
from queue import PriorityQueue

def create_occupancy_grid_from_image(image_path, threshold=128):
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    _, thresholded = cv2.threshold(image, threshold, 255,
                                   cv2.THRESH_BINARY)
    occupancy_grid = thresholded // 255
    return occupancy_grid, image

def heuristic(cell, goal):
    return abs(cell[0] - goal[0]) + abs(cell[1] - goal[1])

def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

def a_star_search(occupancy_grid, start, goal):
    open_set = PriorityQueue()
    open_set.put((0, start))
    came_from = {}
    g_score = {cell: float('inf')
                for cell in np.ndindex(occupancy_grid.shape)}
    g_score[start] = 0

    while not open_set.empty():
        _, current = open_set.get()
        if current == goal:
            return reconstruct_path(came_from, current)

        for dx, dy in [(1,0), (-1,0), (0,1), (0,-1)]:
            x, y = current[0]+dx, current[1]+dy
            if (0 <= x < occupancy_grid.shape[1] and
                0 <= y < occupancy_grid.shape[0] and
                occupancy_grid[y, x] == 1):
                tentative_g = g_score[current] + 1
                if tentative_g < g_score[(x, y)]:
                    came_from[(x, y)] = current
                    g_score[(x, y)] = tentative_g
                    f = tentative_g + heuristic((x,y), goal)
                    open_set.put((f, (x, y)))

    return None

# --- Run ---
image_path = "grid3.png"
start = (350, 50)
```

```

goal = (50, 350)

occupancy_grid, image = create_occupancy_grid_from_image(image_path)
path_found = a_star_search(occupancy_grid, start, goal)

if path_found:
    plt.imshow(occupancy_grid, cmap='gray', origin='lower')
    plt.plot([c[0] for c in path_found],
             [c[1] for c in path_found],
             marker='o', markersize=4, color='blue', label='Path')
    plt.plot(start[0], start[1], 'go', label='Start')
    plt.plot(goal[0], goal[1], 'ro', label='Goal')
    plt.legend()
    plt.title('A* Path on Occupancy Grid')
    plt.grid(True)
    plt.show()
else:
    print('No path found!')

```

Part 12: Exercises — Build It Yourself

Exercise 1: Minimal Pathfinding (Beginner)

Create a 10×10 NumPy grid manually. Place obstacles as walls. Implement A* from scratch and visualize the result using matplotlib.

```
grid = np.ones((10, 10), dtype=int)
grid[3, 2:8] = 0 # horizontal wall
grid[6, 2:8] = 0 # another wall
start = (1, 1)
goal = (8, 8)
# Implement A* on this grid...
```

Exercise 2: Compare BFS vs A* (Intermediate)

Implement both BFS and A* on the same grid. Add a counter to track how many nodes each algorithm expands before finding the path. Visualize the explored regions (hint: color every explored cell in light blue). You should see A* exploring far fewer nodes.

Exercise 3: Diagonal Movement (Intermediate)

Extend the A* code to support 8-directional movement. Update the step cost (diagonal = $\sqrt{2}$, cardinal = 1) and the heuristic (switch from Manhattan to Euclidean distance). Compare the resulting path length and shape with the 4-directional version.

Exercise 4: Trace A* by Hand (Intermediate)

Using the following grid, trace A* step by step on paper. Show the open set, g-scores, and came_from at each iteration. Verify your answer by running the code.

```
S . . . .
. . X . .
. . X . .
. . . . .
. . . . G
```

Exercise 5: Dynamic Obstacles (Advanced)

After finding a path, programmatically add a new obstacle that blocks it. Rerun A* and display both the old path (dashed red) and new path (solid blue) on the same plot.

Exercise 6: Obstacle Inflation (Advanced)

In real robotics, the robot has physical size. Implement obstacle inflation: expand all obstacles by a safety margin (e.g., 5 pixels in every direction) before running A*.

```
import scipy.ndimage

obstacle_mask = (occupancy_grid == 0).astype(np.uint8)
inflated = scipy.ndimage.binary_dilation(
```



```
obstacle_mask, iterations=5)  
safe_grid = np.where(inflated, 0, 1)
```

Exercise 7: Non-Uniform Costs (Advanced)

Create a grid where different cells have different traversal costs (e.g., road = 1, grass = 3, mud = 5). Modify A* to use these costs instead of a uniform cost of 1. Observe how the path prefers lower-cost terrain even if it means a longer route. This is also where you will see g-scores decrease in practice.

Part 13: Summary & Key Takeaways

Core concepts covered:

1. **Occupancy grids** convert real-world maps (images) into a format algorithms can reason about: a binary matrix of free (1) and occupied (0) cells.
2. **Thresholding** is the bridge between a grayscale image and a binary grid. The threshold value determines what counts as free vs occupied.
3. **$A^* = g(n) + h(n)$** — it finds the shortest path by combining actual path cost with an optimistic estimate of remaining cost.
4. **Manhattan distance** is the correct heuristic for 4-directional grid movement because it is admissible (never overestimates).
5. **$h(n)$ ignores obstacles** — it always computes straight-line distance. Obstacles are handled by the occupancy check and g-score tracking, not by the heuristic.
6. **Priority queues** ensure we always expand the most promising node next.
7. **came_from is a tree, not a path.** It stores breadcrumbs to every cell A* discovered, including dead ends and side branches. Entries are never deleted. Reconstruction works by following only the chain from goal back to start, ignoring everything else.
8. **Path reconstruction** follows the came_from chain backwards from goal to start.
9. **Coordinate conventions** (x,y vs row,col) are the most common source of bugs in grid-based pathfinding.

The concept dependency chain:

Image → Threshold → Occupancy Grid → Implicit Graph →
Search (BFS → Dijkstra → A*) →
Heuristics → Priority Queue → Path Reconstruction → Visualization

Each concept builds on the previous one. Master them in order, and you can implement not just A*, but any grid-based pathfinding algorithm from scratch.

Appendix A: Required Libraries

```
pip install numpy opencv-python matplotlib
```

Library	Purpose
numpy	Array operations, grid representation
opencv-python (cv2)	Image loading and thresholding
matplotlib	Visualization and plotting
queue.PriorityQueue	Built-in Python min-heap (no install needed)

Appendix B: Creating Test Images

If you do not have a grid image, create one programmatically:

```
import numpy as np
import cv2

# Create a 400x400 white image
img = np.ones((400, 400), dtype=np.uint8) * 255

# Draw black obstacles (filled rectangles)
cv2.rectangle(img, (100, 50), (120, 300), 0, -1)
cv2.rectangle(img, (200, 100), (220, 350), 0, -1)
cv2.rectangle(img, (300, 50), (320, 250), 0, -1)

# Save
cv2.imwrite('grid3.png', img)
print('Test image created!')
```

Appendix C: PriorityQueue Quick Reference

```
from queue import PriorityQueue

pq = PriorityQueue()

pq.put((5, 'low priority'))    # Add with priority 5
pq.put((1, 'high priority'))  # Add with priority 1

priority, item = pq.get()      # Returns (1, 'high priority')
print(pq.empty())              # False (one item remains)
```

The priority queue always returns the element with the **smallest** first value. This is why we use f-score as the priority — the node with the lowest estimated total cost gets explored first.